PAALHU: project status and pending work

Restaurant website with bookings and orders · 29 October 2026 · Based on the "From Screens to Endpoints" worksheet and the course project structure

1. Summary

Part	Status	Notes
Public screens (home, menu, set menu, spice guide, opening hours, location, about us, booking)	Done	Static HTML. Must move into `public/` (step 2)
Chat widget on the website	Built, off	No AI model connected yet (`CHAT_ENABLED = false`)
Booking logic	Mostly done	In `reservas-core.mjs`, stored in a JSON file. `data/store.js` will share it
Express structure (`app.js`, `server.js`, `middleware/logger.js`, `routes/`)	Done	Commit `bea95cb`. Two small changes left to match the course (step 1)
Data store (`src/data/store.js`)	Not started	In-memory store + seed data. Replaces the SQLite plan
Validators	Written	All in one file, `src/validators.js`. Must be split into `src/validators/<resource>Validator.js`
Endpoints	2 of 16	Only `GET /api/chat/status` and `POST /api/chat`
Orders	Not started	Only a script that prices an order; nothing saves orders
Pagination and filters	Not started	The `page` and `limit` validators already exist
Tests (Jest + supertest)	Not started	`tests/<resource>.test.js`
README.md and .env	Done	Commit `183b101`. README needs updating at the end

> **Agreed decisions.** The project follows the course structure exactly: `app.js`, `server.js`, `middleware/logger.js`, `routes/`, `controllers/`, `data/store.js`, `validators/`, the browser client in `public/` and tests in `tests/`. Data lives in an in-memory store with seed data (no database). There are no `models/`, no `db/`, no separate error-handler file and no staff login. Customers book and order through the chat, by phone (+34 627 41 09 35) or by email (owners@paalhu.es); the chat can only do what the endpoints can do. Validators only say what is wrong and in which field; the controllers and the error handler choose the HTTP status code.

2. Project structure

The whole repository, as it is today and as it has to end up.

● exists ● exists, needs changes ● missing ~~struck out~~ = delete

```
PAALHU/
├ package.json                Add "test" script and the dev dependencies jest and supertest
├ package-lock.json           Exact installed versions (automatic, don't edit)
├ .gitignore                  node_modules, .env, customer data, booking history
├ README.md                   Update paths (public/), endpoints and npm test
├ .env                        PORT (git-ignored)
```

├ src/	The server
│ ├ app.js	Middleware pipeline, routers, 404, error handler **last**. Exports the app (no listen)
│ ├ server.js	Time zone, port, `app.listen` only
│ ├ middleware/	
│ │ ├ logger.js	Method, URL, status and time of each request (Session 15)
~~│ │ └ errorHandler.js~~	Its code moves to the end of `app.js`
│ ├ routes/	Thin routers: URL → controller function
│ │ ├ chat.js	The 2 chat endpoints
│ │ ├ bookings.js	
│ │ ├ waitingList.js	
│ │ ├ orders.js	
│ │ └ dishes.js	
│ ├ controllers/	Handler logic: read the request, validate, call the store, answer
│ │ ├ chatController.js	Chat assistant with `status` and `send`
│ │ ├ bookingController.js	
│ │ ├ waitingListController.js	
│ │ ├ orderController.js	
│ │ └ dishController.js	
│ ├ data/	
│ │ └ store.js	In-memory store + seed data (dishes, orders) and the shared booking engine
│ ├ validators/	Hand-written validation (no Zod yet)
│ │ ├ bookingValidator.js	Split from today's `validators.js`
│ │ ├ orderValidator.js	
│ │ └ commonValidator.js	
~~│ ├ validators.js~~	Replaced by `validators/`
~~│ └ models/~~	Empty; not in the course structure
├ public/	The browser client, served by Express
│ ├ index.html … booking.html	The 8 pages, moved from `pages/`
│ ├ styles.css, chat.css	Moved from `css/`
│ ├ app.js	The chat widget, moved from `js/chat.js`
│ └ img/	Photos (dining room, family)
~~├ pages/, css/, js/, img/~~	Empty once everything is in `public/`
├ tests/	Jest + supertest
│ ├ validators.test.js	
│ ├ bookings.test.js	
│ ├ waitingList.test.js	
│ ├ orders.test.js	
│ └ dishes.test.js	
├ docs/	This document (PDF and its HTML source)
└ .claude/skills/	Tools for Claude and staff (not part of the website)
├ paalhu-assistant/	`SKILL.md`, `reference.md` (menu, prices, hours), `order_total_validator.mjs`
└ paalhu-reservas/	`SKILL.md`, `config.json` (tables, shifts, policy), `reservas-core.mjs` (booking engine), `reservas.mjs` (staff CLI), `evals/`

Course structure compared with ours

Course file	Function	Status	What we need

`src/app.js`	Middleware pipeline, routers, 404, error handler last	Change	Move the error handler in from `middleware/errorHandler.js`; serve `public/`; drop the `/` redirect
`src/server.js`	`app.listen` only	Done	—
`src/middleware/logger.js`	Logs every request	Done	—
`src/routes/<resource>.js`	Thin router	1 of 5	`chat.js` exists; add bookings, waitingList, orders, dishes
`src/controllers/<resource>Controller.js`	Handler logic, calls the store	1 of 5	`chatController.js` exists; add the other four
`src/data/store.js`	In-memory store + seed data	Missing	See step 3
`src/validators/<resource>Validator.js`	Hand-written validation	Change	Split `validators.js`
`public/`	Browser client served by Express	Change	Move `pages/`, `css/`, `js/`, `img/` in
`tests/<resource>.test.js`	Jest + supertest	Missing	See step 8

Endpoints we already have

Endpoint	Function	Where in the code
`GET /api/chat/status`	Tells the website whether the chat is on: `{ enabled }`. The widget only shows itself when it is `true`. Today it is `false`.	`routes/chat.js` → `status()` in `chatController.js`
`POST /api/chat`	Receives `{ session, message }`, returns `{ session, reply }`. 200 reply · 400 empty message or bad JSON · 429 busy or conversation too long · 502 model failed · 503 chat off (always, today).	`routes/chat.js` → `send()`, built on `answer()`

3. What is already done

Screens. All public pages, listing chat, phone and email for bookings and orders. Chat widget on the home and booking pages.

Express (step 1, commit `bea95cb`). `app.js` with logger, `express.json()` (16 KB), the chat router, static files from the four website folders only, JSON 404 under `/api/` and an error handler. `server.js` only starts it. `npm start` reads `.env` if it exists. Tested: pages, CSS, JS and images load; `src/`, `.claude/`, `node_modules/`, `package.json` and `.env` answer 404.

Booking engine (`reservas-core.mjs`). Availability with alternatives when a shift is full; create, change and cancel bookings; automatic table assignment, including adjacent tables; waiting list, shift occupancy, booking sheet, search; permanent history of every change. Used by the chat and by the staff command-line tool.

Entity	Status	Where
Booking, WaitingListEntry	Done	`reservas-core.mjs` → `reservas.json`. Channel stored only as `chat` / `equipo`
HistoryEvent	Done	`reservas-historial.jsonl`
Table, Shift	Done	`config.json`

Dish	Partial	Only as text in `reference.md` and the menu HTML, no ids → seed data in `store.js`
Order, OrderItem	Missing	→ array in `store.js`

Validators (`src/validators.js`). Each returns the clean value or throws a `ValidationError` with a readable message and the wrong field. No HTTP codes inside. They will be split like this:

New file	Validators
`bookingValidator.js`	`date`, `time`, `guests`, `name`, `phone`, `notes`, `table`, `reference`, `channel`
`orderValidator.js`	`dish`, `drink`, `quantity`, `price`, `spiceLevel`, `items`, `type`, `address`, `orderTime`, `orderStatus`, `statusChange`
`commonValidator.js`	`ValidationError`, `page`, `limit` (`login` is no longer needed)

4. What we need to do, in order

Step 1 · Express structure Done + two changes
Move the error handler from `src/middleware/errorHandler.js` to the end of `app.js` (after the 404) and delete the file. Delete the empty `src/models/`. Everything else in step 1 is done.

Step 2 · Browser client in `public/`
Move the 8 pages from `pages/`, `styles.css` and `chat.css` from `css/`, `js/chat.js` as `public/app.js`, and `img/` into `public/`. Fix the links in the pages (`../css/styles.css` → `styles.css`, `../js/chat.js` → `app.js`, `../img/…` → `img/…`). In `app.js`: one `express.static('public')`; `/` then serves `public/index.html` directly, so the redirect goes.

Step 3 · Data store (`src/data/store.js`)
One module that owns all the data, so controllers never touch files.

- **Seed data:** dishes and drinks from `reference.md` with an id, name, category, price in cents and spice level.
- **Orders:** an in-memory array with an id counter. Create, list, get one, change status. The total is calculated here from dish prices.
- **Bookings and waiting list:** the store creates **one** booking engine (`abrirLibro()`) and exports it. The chat, the new controllers and the staff CLI all use the same rules.
- Exports a `reset()` so each test starts from the seed data.

Step 4 · Validators per resource
Split `validators.js` into `src/validators/` as in section 3. Update the imports in `chatController.js`. The chat also starts checking `time`, `notes` and `reference`. Every new controller validates its body, params and query before calling the store.

Step 5 · Controllers and routes (14 new endpoints)

#	Controller	Endpoint	Function
1	bookingController	`GET /api/bookings/shifts?date=`	Shifts of a day and their occupancy
2		`GET /api/bookings/availability?date=&time=&guests=`	Is there a table, and alternatives if not
3		`GET /api/bookings`	List, with filters and pagination
4		`GET /api/bookings/:id`	One booking
5		`POST /api/bookings`	Create (phone or email) → 201
6		`PATCH /api/bookings/:id`	Change or cancel

7	waitingListController	`GET /api/waiting-list`	List, by date and status
8		`POST /api/waiting-list`	Add an entry → 201
9		`PATCH /api/waiting-list/:id`	Update (called, turned into a booking…)
10	orderController	`GET /api/orders`	List, with filters and pagination
11		`GET /api/orders/:id`	One order
12		`POST /api/orders`	Create an order → 201
13		`PATCH /api/orders/:id`	Change status
14	dishController	`GET /api/dishes`	Menu with ids and prices
—	chatController	`GET /api/chat/status`, `POST /api/chat`	Already exist

Shifts and availability now live under `/api/bookings/…` so they fit in `routes/bookings.js`. Cancelling uses `PATCH` with `status: "cancelled"`, never `DELETE`: bookings and orders are never deleted.

Step 6 · Pagination and filters

`GET /api/bookings`: `page`, `limit` (20, max 100), `date` / `from` / `to`, `status`, `search`, `channel`, `sort`. `GET /api/orders`: `page`, `limit`, `date`, `status`, `channel`, `type`, `sort`. `GET /api/waiting-list`: `date`, `status`. Paginated answers return `{ data, page, limit, total }`.

Step 7 · Chat orders

Add a `create_order` tool to the chat that uses the store. Remove the rule that tells customers to call to confirm their order. Connect an AI model so the chat is switched on.

Step 8 · Tests (Jest + supertest)

`tests/validators.test.js` first, then one file per resource that calls the endpoints with `supertest(app)`. This is why `app.js` exports the app without `listen`. Add `jest` and `supertest` as dev dependencies and a `"test"` script.

5. Details to take into account

Server and Express

Point	Detail
Order inside app.js	logger → `express.json()` → `express.static('public')` → routers → 404 → error handler. The error handler goes last and has 4 arguments `(err, req, res, next)`.
Express 5 and async	Errors thrown in async handlers reach the error handler on their own: no try/catch in every route.
Madrid time	`process.env.TZ = 'Europe/Madrid'` stays the first line of `server.js`. Tests must set it too (Jest `setupFiles`), or dates shift.
Keep the chat working	`public/app.js` relies on `GET /api/chat/status` returning `{ enabled }` and on the 503 while the chat is off. Same URLs, same answers.
Private files	Only `public/` is served. `src/`, `.claude/` (customer data), `node_modules/`, `package*.json` and `.env` are never reachable.
Jest with ES modules	The project uses `"type": "module"`. Run Jest as `node --experimental-vm-modules node_modules/jest/bin/jest.js` in the `"test"` script.
Tests and real data	The booking engine writes to `reservas.json`. In tests, set `RESERVAS_DATOS` to a temporary file so tests never touch real bookings.

Errors and status codes

Point	Detail
Who decides the code	Validators throw `ValidationError` (message + field); the store throws its own errors with a `status`. Only the error handler turns them into 400 / 404 / 409 / 500.
Error answer format	Always `{ error, field? }`. On 500, never send internal details (stack, file paths); log them on the server.
Success codes	200 for reads and changes, 201 for a POST that creates something.
Don't leak data	"Not found" must be the same whether the booking doesn't exist or the phone doesn't match (the chat already does this).

Data and privacy

Point	Detail
In-memory store	Orders are lost when the server restarts; that is accepted for the course. Bookings keep their JSON file because the engine already saves them.
Customer data off GitHub	`reservas.json` and `*historial.jsonl` stay in `.gitignore`.
What customers see	Never table numbers, nor other customers' names or phones.
Nothing is deleted	Cancelling = `PATCH status: "cancelled"`. Every booking change is written to the history.
Channel	`chat`, `phone` or `email` (today `chat` / `equipo`). Requests send `phone` or `email`; only the server sets `chat`.
Order total	Calculated in the store from the seed prices; never trust a total or price sent by the client. Work in cents.
Spanish field names	The booking engine uses `fecha`, `hora`, `personas`, `nombre`, `telefono`, `notas`, `estado`. The API uses English; the controller maps them, as `runTool` in `chatController.js` does today.

Business rules

Point	Detail
Group size	Maximum 12 guests (`config.json` → `grupo_maximo`). Bigger groups write to owners@paalhu.es.
Cancellation	Free with at least 24 h notice. No deposit.
Times	Bookings: arrival time inside a shift (`config.json` → `turnos`); the table is held for the whole shift. Orders: inside opening hours.
Set menu	Monday to Friday, lunch only.
Spice level	1 to 5 for every dish in an order.
Allergens	Not published per dish. Never say a dish is safe or free of an allergen; note allergies on the booking and refer to owners@paalhu.es.
Order status	received → confirmed → ready → delivered, or cancelled before delivered. Use `statusChange`.

Quality

Point	Detail

README.md	Install (`npm install`), run (`npm start`), test (`npm test`), an example `.env` , and the list of endpoints.
Language	Code, comments and API in English; the booking engine keeps its Spanish names. Customer messages in the customer's language.
Commits	Small commits with a message saying what changed (not just "commit").

6. Open points to decide

Point	Detail
No staff login	The course structure has no login, so every endpoint is public. That means anyone could list bookings with names and phones. Decide whether that is acceptable for the course, or whether list endpoints should hide phones.
Worksheet vs. this plan	The entities sheet (`Book.xlsx`) has customer sign-up and login, an allergen filter, Table as its own entity, and `/api/categories` , `/api/drinks` , `/api/menus` . This plan keeps the earlier decisions (no customer accounts, no allergen data, tables assigned by the engine). Decide which endpoints from the sheet to add.
Maximum guests	`guests` accepts 1–14, but the restaurant's maximum is 12. Change the validator to 12.
Same-day bookings	The `date` validator rejects today, so "a table for tonight" fails. The worksheet says only "not in the past". Decide which rule applies.
Hours in several places	Opening hours are in `reference.md` , the website and `orderValidator.js` ; shifts in `config.json` . Decide whether to move them all into `config.json` .
File rename	`order_total.mjs` was renamed to `order_total_validator.mjs` , but `SKILL.md` , `reference.md` and the script's usage text still use the old name.